

AUDITORÍA TÉCNICA

motor-contactos

Lead Systems Architect Review — 2026-08-14 — Repositorio privado, no distribuir

1. Executive Summary

motor-contactos resuelve un problema concreto y ya resuelto en producción: unificar los contactos reales de dos cuentas de Google (Pablo y Sindy), acumulados durante más de 15 años con formatos inconsistentes, duplicados y campos mal cargados, en una única lista maestra confiable y sincronizable de vuelta a Google Contacts.

Estado real de los datos, verificado en la base al momento de esta auditoría:

Métrica	Valor
Registros crudos importados	36.103
Registros normalizados	36.102
Contactos finales (deduplicados)	8.541
Casos pendientes de revisión manual	0

El proyecto NO es un prototipo ni un MVP a medio camino: tiene pipeline de datos completo, deduplicación con salvaguardas encontradas contra datos reales, dos interfaces funcionales (app de escritorio empaquetada y panel web), automatización mensual sin supervisión, e integraciones reales (Google, HubSpot/Mailchimp/Brevo, WhatsApp vía MejoraWS). Lo que sigue abierto (sección 6) son 2 pasos que requieren login humano de Google, no deuda de desarrollo.

Suite de tests al momento de esta auditoría: 195 passed in 14.71s — todos en verde.

2. Arquitectura & Mapa de Archivos

2.1 Stack real (versiones leídas de requirements.txt / package.json)

Capa	Tecnología	Versión mínima declarada
Backend	Python	3.14 (venv del proyecto)
Backend web	Flask	3.1
Datos	SQLite (WAL) vía sqlite3 stdlib	—
Excel	openpyxl / pandas / xlrd / odfpy	3.1 / 3.0 / 2.0 / 1.4
Teléfonos	phonenumbers	9.0
Fuzzy matching	rapidfuzz	3.14
Extracción PDF/DOCX/HTML	pdfplumber / python-docx / BeautifulSoup4+ lxml	0.11 / 1.2 / 4.13+5.3
OCR	pytesseract + Pillow (requiere binario Tesseract aparte)	0.3 / 11.0
Google	google-api-python-client + google-auth(-oauthlib)	2.149 / 2.35 / 1.2
App de escritorio	pywebview + PyInstaller (build-only)	6.2 / 6.22
Frontend	React + TypeScript + Vite	18.3 / 5.6 / 5.4
UI: estilos	Tailwind CSS	3.4.13
UI: listas grandes	@tanstack/react-virtual	3.10
UI: modal	@radix-ui/react-dialog	1.1
Tests	pytest	9.0

2.2 Mapa de módulos (src/motor/) — qué hace cada uno REALMENTE

Módulo	Responsabilidad real
staging_db.py	Esquema SQLite (9 tablas) + migraciones ALTER TABLE incrementales sobre la base ya existente.
ingest.py	Orquesta los extractores de archivo sobre Data/Crudos/, con try/except por archivo (uno roto no frena el resto).
google_contacts_source.py	Import en vivo desde People API (contactos + "otros contactos"), incremental por etag.
extractors/*.py	Un extractor por formato (CSV, Excel, VCF, JSON, HTML, DOCX, PDF, OCR, texto libre) + column_mapping.py compartido (alias ES/EN/HubSpot/Mailchimp/Brevo).
text_cleaning.py	Heurísticas de limpieza calibradas contra datos reales rotos (teléfono guardado como nombre, honoríficos, cargos concatenados, filas plantilla).
normalize_pipeline.py	Conecta raw_records con phone_normalizer/email_normalizer/text_cleaning/tagging, escribe normalized_records.
phone_normalizer.py / area_codes_ar.py	Normalización E.164 con reglas específicas de Argentina (ver sección 4.1).
email_normalizer.py	Normalización + corrección de dominios comunes mal tipeados (distancia de Levenshtein).
tagging.py	Auto-etiquetado (familiar/laboral/cliente/proveedor/personal) por palabras clave.
dedup/blocking.py, scoring.py, union_find.py, learning.py	Motor de deduplicación (ver sección 4.2).
llm_judge.py	Cliente IA escalonado (Grok → OpenRouter gratis → Anthropic pago) para la banda dudosa.
merge_engine.py	Aplica las 3 bandas, materializa clusters, deshacer puntual/total.
anomalias.py	Detección de teléfonos compartidos por +5 contactos finales distintos.
export.py	Lista maestra .xlsx, CSV para WhatsApp (MejoraWS), API de búsqueda/edición/listado.
api.py / reviewer_app.py	API JSON (para la UI React) y panel HTML clásico — mismo motor, dos interfaces.
desktop_app.py	Envuelve Flask + la UI React compilada en una ventana nativa (pywebview).
cli.py	Punto de entrada único (`python -m motor.cli <comando>`) para todos los comandos.

2.3 Pipeline de datos de punta a punta

- 1. Entrada: People API de Google (en vivo) o archivos sueltos en Data/Crudos/ (CSV/Excel/VCF/JSON/HTML/DOCX/PDF/OCR/texto libre) → raw_records (inmutable).
- 2. Normalización: phone_normalizer + email_normalizer + text_cleaning + tagging → normalized_records.
- 3. Bloqueo (blocking.py): agrupa candidatos a comparar por teléfono/email exacto o clave fonética apellido+inicial — evita comparar todos contra todos.
- 4. Clasificación (scoring.py): score 0–1 por señales, 3 bandas de confianza (ver 4.2).
- 5. Fusión (merge_engine.py): materializa clusters vía union-find, nunca destructivo.
- 6. Salida: lista-maestra.xlsx, CSV para WhatsApp, API para las dos interfaces, Apps Script para sync de vuelta a Google.

3. Estado de la interfaz

El proyecto NO es CLI pura: existen dos interfaces visuales completas y funcionales, construidas sobre la misma API/motor.

3.1 App de escritorio (App/MotorContactos.exe, pywebview + React)

- Ventana nativa de Windows, sin terminal ni pestaña de navegador — identidad visual de Mejora Continua (tipografía, paleta, logo).

- Sidebar con 3 secciones (Contactos / Revisión pendiente / Sync a Google), 4 tarjetas de métricas en vivo.
- Tabla de contactos: 12 columnas configurables (mostrar/ocultar) y redimensionables (drag), columna Nombre fija a la izquierda, filtro "contiene" por columna + multiselect en Tag, búsqueda global instantánea client-side (toda la base cargada en memoria, sin ida y vuelta al servidor), filas virtualizadas (@tanstack/react-virtual — renderiza solo lo visible aunque haya miles de filas).
- Cola de revisión en lote con contexto completo por par (nombre, organización, teléfono, email, fuente, foto) — no un id numérico pelado.
- Editor de contacto en modal, todos los campos editables, la corrección manual pisa el cálculo automático de forma no destructiva (tabla ediciones_manuales aparte).

3.2 Panel clásico (Iniciar Panel.bat, HTML servido por Flask)

- Mismo motor, mismas acciones (correr pipeline, revisar, buscar/editar, exportar), sin la identidad visual nueva ni las columnas configurables — pensado como respaldo de un solo proceso si la app de escritorio fallara.

3.3 Lo que falta pulir (autocrítica del propio equipo, no genérica)

- Los popovers de filtro de columnas muy a la derecha (Tag, Nota) pueden abrirse fuera del viewport visible en tablas muy anchas.
- Los colores de avatar son una paleta genérica de Tailwind, no derivada de la paleta de marca.

4. Lógica de negocio

4.1 Normalización de teléfono — casos límite reales que maneja

- Formato de salida target: +549 + 10 dígitos (celular) / +54 + 10 dígitos (fijo) — estilo WhatsApp.
- Detecta y separa números pegados sin separador (bloques de 10 dígitos consecutivos, ≥ 20 dígitos totales).
- Detecta notación científica de Excel ("3,76505E+09") ANTES de partir por separadores — no se "completa" con un código de área default (fabricaría un número falso), va directo a revisión.
- Prioriza SIEMPRE la interpretación doméstica argentina sobre la detección global de país cuando el patrón encaja (10 dígitos, 9+10, 54+10/11, 6–8 dígitos) — un bloque de dígitos "pelado" puede coincidir por casualidad con el plan de numeración de otro país.
- Ambigüedad celular/fijo sin ninguna pista: usa el default configurado y deja la bandera 'movil-asumido' — nunca decide en silencio sin dejar rastro.
- Números cortos (6–8 dígitos): se completan con el código de área default, quedan marcados 'incompleto'+ 'corregido'.
- Múltiples números en un mismo campo (separados por ; , / | "y" o ':::') se dividen y deduplican por E.164 resultante.

4.2 Criterio de deduplicación

Bloqueo (evita $O(n^2)$): candidatos agrupados por teléfono exacto, email exacto, o clave fonética apellido+inicial de nombre; bloques de más de 500 elementos se descartan para no explotar en combinatoria (esos casos igual se capturan si comparten teléfono/email exacto).

Score y bandas (config.yaml $\rightarrow$ dedup):

- $\text{score} \geq 0.80 \rightarrow$ fusión automática ("trigo").
- $\text{score} \leq 0.55 \rightarrow$ separación automática ("paja").
- $0.55 < \text{score} < 0.80 \rightarrow$ "zona gris": IA escalonada (Groq $\rightarrow$ modelos gratis de OpenRouter $\rightarrow$ Anthropic pago) y, si nadie resuelve con confianza suficiente, cola de revisión humana en lote.

- Pesos del score cuando no hay señal exacta: teléfono 0.5, email 0.3, similitud de nombre 0.15, similitud de organización 0.05.
- Salvaguarda real (encontrada corriendo contra los datos reales, no teórica): teléfono/email exacto normalmente fuerza score=1.0, EXCEPTO si ambos contactos traen nombre completo y son claramente distintos entre sí (similitud < 0.5) — ahí se asume línea compartida (familia/oficina) y va a zona gris en vez de fusionar en silencio.
- Aprendizaje: cada decisión humana ajusta (hasta ± 0.1 , con mínimo 15 casos de evidencia) el score futuro de ese patrón de señales específico — no es ML, es una tasa de aceptación auditable.

4.3 Integraciones — estado real, no aspiracional

Integración	Estado real
Google Contacts (import)	Funcionando en producción con datos reales.
Google Contacts (sync de vuelta, Apps Script)	Código completo y migrado a People API — NUNCA probado en vivo, requiere login humano de Google.
"Otros contactos" de Gmail	Código completo — nunca probado en vivo, requiere un login humano APARTE (scope OAuth distinto).
WhatsApp (MejoraWS)	Funcionando: export en el formato exacto que MejoraWS espera, verificado contra datos reales.
HubSpot / Mailchimp / Brevo	Funcionando vía reconocimiento de encabezados CSV — no hay integración por API con esas plataformas (no se generaron API keys).

5. Diagnóstico de puntos críticos

5.1 Deuda técnica real (no genérica)

- El servidor de la app de escritorio y del panel es el servidor de desarrollo de Flask (Werkzeug) — Flask mismo advierte no usarlo en producción. Riesgo bajo en este contexto (un solo usuario local, nunca expuesto a internet), pero es una dependencia real a tener presente si algún día se expone la app fuera de localhost.
- Backup exclusivamente local (repo git sin remoto en esta misma PC) — es punto único de falla: si el disco de esta máquina falla, no hay copia offsite de Data/Salida/staging.sqlite.
- App/MotorContactos.exe (PyInstaller) no se reconstruye automáticamente al cambiar el código — nada impide que quede desactualizado si se toca ui/ o el backend y no se corre scripts/build_exe.ps1 antes de usarlo.
- OCR de capturas/imágenes depende de que Tesseract-OCR (el binario) esté instalado aparte del proyecto — si falta, el extractor devuelve lista vacía sin avisar explícitamente al usuario final en la interfaz.
- Sin pipeline de CI para motor-contactos (a diferencia de la SPA hermana del repo, que sí corre GitHub Actions) — los 195 tests solo se ejecutan cuando alguien los corre a mano.

5.2 Cuellos de botella de rendimiento

Con el volumen real actual (8.541 contactos finales. 36.103 crudos) no hay cuello de botella medido: la UI carga toda la base en memoria y filtra client-side sin demora perceptible, el blocking evita la explosión combinatoria. El único techo conocido es el tope de 500 elementos por bloque de blocking — un apellido extremadamente común por encima de ese tope pierde blocking por nombre (igual se captura por teléfono/email exacto).

5.3 Riesgos operativos

⚠ Ningún paso de escritura hacia Google Contacts (Sync.gs) fue probado contra una cuenta real todavía — el primer uso real debería ser contra un Sheet de prueba, no la lista completa.

⚠ credentials.json y los token_*.json de Google están gitignoreados correctamente, pero viven sin cifrado en disco — cualquier proceso con acceso a esta cuenta de Windows podría leerlos.

⚠ No hay backup automático PRE-escritura antes de una corrida de deduplicación grande — el mecanismo de "deshacer" cubre el caso de arrepentimiento, pero depende de que alguien lo accione a tiempo.

6. Roadmap

Pendientes reales según el código y PENDIENTES.md — no una lista genérica de buenas prácticas:

- Probar Sync.gs contra una cuenta de Google de PRUEBA (2-3 contactos ficticios) antes de correrlo contra las cuentas reales de Pablo y Sindy — requiere login humano, bloqueado hasta que se haga.
- Si Sync.gs funciona en la prueba: correrlo contra las cuentas reales.
- Probar "otros contactos" (importar-otros-contactos) en vivo — requiere un segundo login humano con un scope OAuth distinto al de Sync.gs.
- Arreglar el posicionamiento de los popovers de filtro para columnas del extremo derecho de la tabla (ver 3.3).
- Derivar la paleta de colores de avatar de la identidad de marca en vez de la paleta genérica de Tailwind (ver 3.3).
- Evaluar backup offsite/en la nube de Data/ — hoy el único respaldo vive en la misma máquina (ver 5.3).
- (Explícitamente pospuesto, no en este roadmap salvo pedido nuevo del usuario: escaneo de directorios completos de la PC, integración directa por API con HubSpot/Mailchimp/Brevo, agente autónomo más allá de la automatización mensual ya construida.)